TechDF Análise SQL

Validações, base limpa e KPIs do CRISP-DM

Ciência de Dados e Aprendizagem de Máquina (A1) · UDF
Profa. Kadidja Valéria

Gustavo Carvalho de Oliveira (P1 e P2)

Dados simulados de uma loja on-line fictícia de informática no DF: 1.200 clientes e 4.776 pedidos (jan./2025 a jun./2026), com erros inseridos de propósito.



-- V1. Campos obrigatórios nulos

```sql
SELECT COUNT(*) AS nulos
FROM "Pedido"
WHERE "idCliente" IS NULL OR "dataPedido" IS NULL
   OR "idFormaPagamento" IS NULL OR "status" IS NULL
```

	nulos
1	0



-- V2. Itens com quantidade zero ou negativa

```sql
SELECT COUNT(*) AS itens_quantidade_invalida
FROM "ItemPedido"
WHERE "quantidade" <= 0
```

	itens_quantidade_invalida
1	25



-- V3. Pedidos com data futura (extração em 30/06/2026)

```sql
SELECT COUNT(*) AS datas_futuras
FROM "Pedido"
WHERE "dataPedido" > DATE '2026-06-30'
```

	datas_futuras
1	30



-- V4. Provável vírgula esquecida: preço pago > 10x o preço de lista do produto

```
SELECT COUNT(*) AS precos_suspeitos,
       ROUND(SUM(i."quantidade" * i."precoUnitario"), 2) AS valor_inflado
FROM "ItemPedido" i
JOIN "Produto" p ON p."idProduto" = i."idProduto"
WHERE i."precoUnitario" > 10 * p."precoLista"
```

	precos_suspeitos	valor_inflado
1	142	20.322.776,00



-- V5. Pedidos duplicados: mesmo cliente, data, pagamento e mesma lista de itens (duplo clique)

```
WITH assinatura AS (
    SELECT pe."idPedido", pe."idCliente", pe."dataPedido", pe."idFormaPagamento",
           string_agg(CAST(i."idProduto" AS VARCHAR) || 'x' || CAST(i."quantidade" AS VARCHAR),
','
                      ORDER BY i."idProduto", i."quantidade") AS itens
    FROM "Pedido" pe
    JOIN "ItemPedido" i ON i."idPedido" = pe."idPedido"
    WHERE pe."dataPedido" <= DATE '2026-06-30'
    GROUP BY 1, 2, 3, 4
)
SELECT COUNT(*) - COUNT(DISTINCT ("idCliente", "dataPedido", "idFormaPagamento", itens)) AS
pedidos_duplicados
FROM assinatura
```

	pedidos_duplicados
1	45



-- V6. Status incoerente: pedido cancelado com avaliação registrada

```
SELECT COUNT(*) AS cancelados_com_avaliacao
FROM "Pedido"
WHERE "status" = 'Cancelado' AND "avaliacao" IS NOT NULL
```

	cancelados_com_avaliacao
1	20



-- View pedido_base — remove datas futuras e duplicatas e limpa a avaliação de cancelados (Fase 3)

```
CREATE VIEW pedido_base AS
WITH assinatura AS (
    SELECT pe."idPedido", pe."idCliente", pe."dataPedido", pe."idFormaPagamento",
           string_agg(CAST(i."idProduto" AS VARCHAR) || 'x' || CAST(i."quantidade" AS VARCHAR),
','
                      ORDER BY i."idProduto", i."quantidade") AS itens
    FROM "Pedido" pe
    JOIN "ItemPedido" i ON i."idPedido" = pe."idPedido"
    WHERE pe."dataPedido" <= DATE '2026-06-30'
    GROUP BY 1, 2, 3, 4
),
numerado AS (
    SELECT "idPedido",
           ROW_NUMBER() OVER (PARTITION BY "idCliente", "dataPedido", "idFormaPagamento", itens
                              ORDER BY "idPedido") AS ordem
    FROM assinatura
)
SELECT pe."idPedido", pe."idCliente", pe."dataPedido", pe."idFormaPagamento", pe."status",
       pe."prazoEntregaDias",
       CASE WHEN pe."status" = 'Cancelado' THEN NULL ELSE pe."avaliacao" END AS "avaliacao",
       pe."valorFrete"
FROM "Pedido" pe
JOIN numerado n ON n."idPedido" = pe."idPedido" AND n.ordem = 1
```

A view não retorna uma tabela: ela é a base consultada pelas próximas páginas.

-- View item_limpo — remove quantidade inválida e corrige preço sem vírgula (Fase 3)

```
CREATE VIEW item_limpo AS
SELECT i."idItem", i."idPedido", i."idProduto", p."idCategoria", i."quantidade",
       CASE WHEN i."precoUnitario" > 10 * p."precoLista" THEN ROUND(i."precoUnitario" / 100, 2)
            ELSE i."precoUnitario" END AS "precoUnitario"
FROM "ItemPedido" i
JOIN "Produto" p ON p."idProduto" = i."idProduto"
JOIN pedido_base pb ON pb."idPedido" = i."idPedido"
WHERE i."quantidade" > 0
```

A view não retorna uma tabela: ela é a base consultada pelas próximas páginas.



-- View pedido_limpo — base limpa com subtotal e indicador de atraso, usada pelos KPIs

```
CREATE VIEW pedido_limpo AS
SELECT pb.*, s.subtotal,
       CASE WHEN pb."prazoEntregaDias" > 5 THEN 1 ELSE 0 END AS atrasado
FROM pedido_base pb
JOIN (SELECT "idPedido", SUM("quantidade" * "precoUnitario") AS subtotal
      FROM item_limpo GROUP BY 1) s ON s."idPedido" = pb."idPedido"
```

A view não retorna uma tabela: ela é a base consultada pelas próximas páginas.



-- KPI 1. Faturamento, pedidos entregues, ticket médio e taxa de atraso

```
SELECT ROUND(SUM(subtotal), 2)            AS faturamento,
       COUNT(*)                           AS pedidos_entregues,
       ROUND(AVG(subtotal), 2)            AS ticket_medio,
       ROUND(100.0 * AVG(atrasado), 1)    AS perc_atrasados
FROM pedido_limpo
WHERE "status" = 'Entregue'
```

	faturamento	pedidos_entregues	ticket_medio	perc_atrasados
1	12.954.026,59	4.300	3.012,56	26,30



-- KPI 2. Faturamento e pedidos por mês

```sql
SELECT CAST(date_trunc('month', "dataPedido") AS DATE) AS mes,
       COUNT(*) AS pedidos,
       ROUND(SUM(subtotal), 2) AS faturamento
FROM pedido_limpo
WHERE "status" = 'Entregue'
GROUP BY 1
ORDER BY 1
```

	mes	pedidos	faturamento
1	2025-01-01 00:00:00	51	143.647,17
2	2025-02-01 00:00:00	96	309.316,73
3	2025-03-01 00:00:00	112	437.412,56
4	2025-04-01 00:00:00	178	441.671,43
5	2025-05-01 00:00:00	187	462.740,72
6	2025-06-01 00:00:00	195	627.000,01
7	2025-07-01 00:00:00	222	628.969,03
8	2025-08-01 00:00:00	215	676.002,84
9	2025-09-01 00:00:00	287	933.262,66
10	2025-10-01 00:00:00	294	825.958,98
11	2025-11-01 00:00:00	308	895.857,31
12	2025-12-01 00:00:00	320	1.201.628,42
13	2026-01-01 00:00:00	310	957.402,06
14	2026-02-01 00:00:00	305	775.708,97
15	2026-03-01 00:00:00	402	1.210.896,33
16	2026-04-01 00:00:00	296	931.956,34
17	2026-05-01 00:00:00	277	767.240,85
18	2026-06-01 00:00:00	245	727.354,18



-- KPI 3. Top 5 categorias por faturamento (participação no total)

```
SELECT c."categoria",
       ROUND(SUM(i."quantidade" * i."precoUnitario"), 2) AS faturamento,
       ROUND(100.0 * SUM(i."quantidade" * i."precoUnitario")
             / SUM(SUM(i."quantidade" * i."precoUnitario")) OVER (), 1) AS perc_faturamento
FROM item_limpo i
JOIN pedido_limpo pl ON pl."idPedido" = i."idPedido" AND pl."status" = 'Entregue'
JOIN "Categoria" c ON c."idCategoria" = i."idCategoria"
GROUP BY 1
ORDER BY faturamento DESC
LIMIT 5
```

	categoria	faturamento	perc_faturamento
1	Notebooks	4.788.473,40	37,00
2	Celulares	3.485.231,35	26,90
3	Monitores	1.947.814,00	15,00
4	Games	1.538.185,60	11,90
5	Periféricos	405.840,71	3,10



-- KPI 4. Participação de cada forma de pagamento no faturamento

```sql
SELECT fp."formaPagamento",
       COUNT(*) AS pedidos,
       ROUND(100.0 * SUM(pl.subtotal) / SUM(SUM(pl.subtotal)) OVER (), 1) AS perc_faturamento
FROM pedido_limpo pl
JOIN "FormaPagamento" fp ON fp."idFormaPagamento" = pl."idFormaPagamento"
WHERE pl."status" = 'Entregue'
GROUP BY 1
ORDER BY 3 DESC
```

	formaPagamento	pedidos	perc_faturamento
1	Cartão de crédito	1.921	50,20
2	Pix	1.406	30,80
3	Cartão de débito	464	10,80
4	Boleto	509	8,20



-- KPI 5. Prazo médio e percentual de pedidos atrasados por região (hipótese H2)

```sql
SELECT r."regiao",
       COUNT(*) AS entregas,
       ROUND(AVG(pl."prazoEntregaDias"), 1) AS prazo_medio_dias,
       ROUND(100.0 * AVG(pl.atrasado), 1)   AS perc_atrasados
FROM pedido_limpo pl
JOIN "Cliente" cl ON cl."idCliente" = pl."idCliente"
JOIN "Regiao"  r  ON r."idRegiao"   = cl."idRegiao"
WHERE pl."status" = 'Entregue'
GROUP BY 1
ORDER BY 3
```

	regiao	entregas	prazo_medio_dias	perc_atrasados
1	Plano Piloto	873	3,10	7,70
2	Guará	451	3,10	8,20
3	Águas Claras	588	3,70	10,00
4	Taguatinga	838	4,10	13,70
5	Sobradinho	235	5,30	29,40
6	Samambaia	341	5,80	40,20
7	Ceilândia	643	6,30	55,80
8	Gama	331	7,30	86,40



-- KPI 6. Avaliação média por faixa de prazo de entrega

```sql
SELECT CASE WHEN "prazoEntregaDias" <= 3 THEN '1. até 3 dias'
            WHEN "prazoEntregaDias" <= 5 THEN '2. 4 a 5 dias'
            WHEN "prazoEntregaDias" <= 7 THEN '3. 6 a 7 dias'
            ELSE '4. 8 dias ou mais' END AS faixa_prazo,
       COUNT(*) AS pedidos_avaliados,
       ROUND(AVG("avaliacao"), 2) AS avaliacao_media
FROM pedido_limpo
WHERE "status" = 'Entregue' AND "avaliacao" IS NOT NULL
GROUP BY 1
ORDER BY 1
```

	faixa_prazo	pedidos_avaliados	avaliacao_media
1	1. até 3 dias	1.499	4,94
2	2. 4 a 5 dias	1.275	4,85
3	3. 6 a 7 dias	645	4,29
4	4. 8 dias ou mais	350	2,56



-- KPI 7. Recompra em até 90 dias após a primeira compra, por atraso na primeira entrega (hipótese H1)

```
WITH primeira AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY "idCliente" ORDER BY "dataPedido", "idPedido") AS n
    FROM pedido_limpo
    WHERE "status" = 'Entregue'
)
SELECT CASE WHEN f.atrasado = 1 THEN 'Primeira entrega atrasada' ELSE 'Primeira entrega no prazo'
END AS primeira_entrega,
       COUNT(*) AS clientes,
       ROUND(100.0 * AVG(CASE WHEN EXISTS (
                 SELECT 1 FROM pedido_limpo o
                 WHERE o."idCliente" = f."idCliente" AND o."idPedido" <> f."idPedido"
                   AND o."dataPedido" > f."dataPedido"
                   AND o."dataPedido" <= f."dataPedido" + INTERVAL '90 days') THEN 1 ELSE 0 END),
1) AS perc_recompra_90d
FROM primeira f
WHERE f.n = 1 AND f."dataPedido" <= DATE '2026-03-31'
GROUP BY 1
ORDER BY 1
```

	primeira_entrega	clientes	perc_recompra_90d
1	Primeira entrega atrasada	330	46,10
2	Primeira entrega no prazo	817	64,00



-- A. Tabela RFM por cliente na data da extração (base da P1 — o notebook completa ticket e mix de produtos)

```
SELECT pl."idCliente",
       DATE '2026-06-30' - MAX(pl."dataPedido") AS recencia_dias,
       COUNT(*) AS frequencia,
       ROUND(SUM(pl.subtotal), 2) AS valor_total,
       ROUND(AVG(pl."prazoEntregaDias"), 1) AS prazo_medio
FROM pedido_limpo pl
WHERE pl."status" = 'Entregue'
GROUP BY 1
ORDER BY valor_total DESC
LIMIT 10
```

	idCliente	recencia_dias	frequencia	valor_total	prazo_medio
1	366	27	10	100.289,15	4,00
2	510	7	18	84.807,95	4,00
3	726	16	21	82.378,25	2,60
4	606	21	6	68.397,45	4,80
5	1.047	70	17	68.082,26	2,70
6	764	5	17	65.156,15	2,20
7	1.159	133	2	64.115,25	6,50
8	281	33	17	63.850,23	4,40
9	671	48	13	60.288,13	3,10
10	9	195	11	60.074,05	4,60



-- B. Hipótese H3: frequência de compra e ticket médio por participação no programa de fidelidade

```sql
SELECT CASE WHEN c."programaFidelidade" THEN 'Participa' ELSE 'Não participa' END AS fidelidade,
       COUNT(DISTINCT c."idCliente") AS clientes,
       ROUND(1.0 * COUNT(*) / COUNT(DISTINCT c."idCliente"), 2) AS pedidos_por_cliente,
       ROUND(AVG(pl.subtotal), 2) AS ticket_medio
FROM pedido_limpo pl
JOIN "Cliente" c ON c."idCliente" = pl."idCliente"
WHERE pl."status" = 'Entregue'
GROUP BY 1
ORDER BY 1
```

	fidelidade	clientes	pedidos_por_cliente	ticket_medio
1	Não participa	901	3,22	3.045,39
2	Participa	280	4,99	2.944,28



Obrigado!

TechDF · Análise SQL · Gustavo Carvalho de Oliveira

